

SOURCE CODE:

```
#include <stdio.h>

#define MAX 100

struct Patient
{
    int id;
    int severity;
    int arrival;
};

struct Patient patient[MAX];

int count = 0;
int arrivalCount = 0;

/* Find the highest-priority patient */
int findPriority()
{
    int i;
    int best = 0;

    for (i = 1; i < count; i++)
    {
        /* Lower severity number = higher priority */
        if (patient[i].severity < patient[best].severity)
        {
            best = i;
        }
    }
}
```

```

/* Same severity -> FIFO */
else if (patient[i].severity == patient[best].severity &&
        patient[i].arrival < patient[best].arrival)
{
    best = i;
}
}

return best;
}

```

```

/* Register a new patient */

```

```

void addPatient()

```

```

{
    if (count == MAX)
    {
        printf("\nQueue is full!\n");
        return;
    }

```

```

    printf("\nEnter Patient ID: ");
    scanf("%d", &patient[count].id);

```

```

    printf("Enter Severity Level (1-5): ");
    scanf("%d", &patient[count].severity);

```

```

    if (patient[count].severity < 1 ||
        patient[count].severity > 5)
    {
        printf("Invalid severity level!\n");
        return;
    }

```

```

    }

    arrivalCount++;
    patient[count].arrival = arrivalCount;

    count++;

    printf("\nPatient registered successfully!\n");
}

/* Display patients according to priority */
void display()
{
    int i, j;
    struct Patient temp;

    if (count == 0)
    {
        printf("\nNo patients waiting.\n");
        return;
    }

    /* Sort according to severity and FIFO */
    for (i = 0; i < count - 1; i++)
    {
        for (j = i + 1; j < count; j++)
        {
            if (patient[j].severity < patient[i].severity ||
                (patient[j].severity == patient[i].severity &&
                 patient[j].arrival < patient[i].arrival))
            {

```

```

        temp = patient[i];
        patient[i] = patient[j];
        patient[j] = temp;
    }
}
}

```

```

printf("\n===== PATIENT QUEUE =====\n");

```

```

printf("ID\tSeverity\tArrival No.\n");

```

```

for (i = 0; i < count; i++)
{
    printf("%d\t%d\t%d\n",
        patient[i].id,
        patient[i].severity,
        patient[i].arrival);
}
}

```

```

/* Show the next patient */

```

```

void nextPatient()
{
    int index;

    if (count == 0)
    {
        printf("\nNo patients waiting.\n");
        return;
    }
}

```

```

index = findPriority();

printf("\n===== NEXT PATIENT =====\n");

printf("Patient ID : %d\n", patient[index].id);
printf("Severity  : %d\n", patient[index].severity);
printf("Arrival No.: %d\n", patient[index].arrival);
}

/* Treat and remove the highest-priority patient */
void treatPatient()
{
    int index;
    int i;

    if (count == 0)
    {
        printf("\nNo patients waiting.\n");
        return;
    }

    index = findPriority();

    printf("\nPatient treated successfully!\n");

    printf("Patient ID: %d\n", patient[index].id);
    printf("Severity: %d\n", patient[index].severity);

    /* Remove patient */
    for (i = index; i < count - 1; i++)
    {

```

```

        patient[i] = patient[i + 1];
    }

    count--;
}

/* Search patient */
void searchPatient()
{
    int id;
    int i;

    printf("\nEnter Patient ID to search: ");
    scanf("%d", &id);

    for (i = 0; i < count; i++)
    {
        if (patient[i].id == id)
        {
            printf("\nPatient Found!\n");

            printf("Patient ID : %d\n", patient[i].id);
            printf("Severity   : %d\n", patient[i].severity);
            printf("Arrival No.: %d\n", patient[i].arrival);

            return;
        }
    }

    printf("\nPatient not found.\n");
}

```

```

/* Update patient information */
void updatePatient()
{
    int id;
    int i;

    printf("\nEnter Patient ID to update: ");
    scanf("%d", &id);

    for (i = 0; i < count; i++)
    {
        if (patient[i].id == id)
        {
            printf("Enter New Severity Level (1-5): ");
            scanf("%d", &patient[i].severity);

            if (patient[i].severity < 1 ||
                patient[i].severity > 5)
            {
                printf("Invalid severity level!\n");
                return;
            }

            printf("\nPatient information updated successfully!\n");

            return;
        }
    }

    printf("\nPatient not found.\n");

```

```
}
```

```
/* Main function */
```

```
int main()
```

```
{
```

```
    int choice;
```

```
    do
```

```
    {
```

```
        printf("\n\n=====\\n");
```

```
        printf("  EMERGENCY PATIENT MANAGEMENT SYSTEM\\n");
```

```
        printf("=====\\n");
```

```
        printf("1. Register New Patient\\n");
```

```
        printf("2. Display Patient Queue\\n");
```

```
        printf("3. Show Next Patient\\n");
```

```
        printf("4. Treat Patient\\n");
```

```
        printf("5. Search Patient\\n");
```

```
        printf("6. Update Patient\\n");
```

```
        printf("7. Exit\\n");
```

```
        printf("\nEnter your choice: ");
```

```
        scanf("%d", &choice);
```

```
        switch (choice)
```

```
        {
```

```
            case 1:
```

```
                addPatient();
```

```
                break;
```

```
            case 2:
```



```
        display();
        break;

    case 3:
        nextPatient();
        break;

    case 4:
        treatPatient();
        break;

    case 5:
        searchPatient();
        break;

    case 6:
        updatePatient();
        break;

    case 7:
        printf("\nExiting system...\n");
        break;

    default:
        printf("\nInvalid choice! Try again.\n");
}

} while (choice != 7);

return 0;
}
```